

GPU INSTANCING & DRAW CALL OPTIMIZATION IN UNITY

**CMPE 485 TERM PROJECT
OPTION A (SINGLE CHALLENGE)
ERAY YÜKLÜ**

***MEASURING HOW DRAW-CALL BATCHING STRATEGIES HOLD UP
AS OBJECT COUNT AND MESH COMPLEXITY GROW ON APPLE
SILICON METAL.***



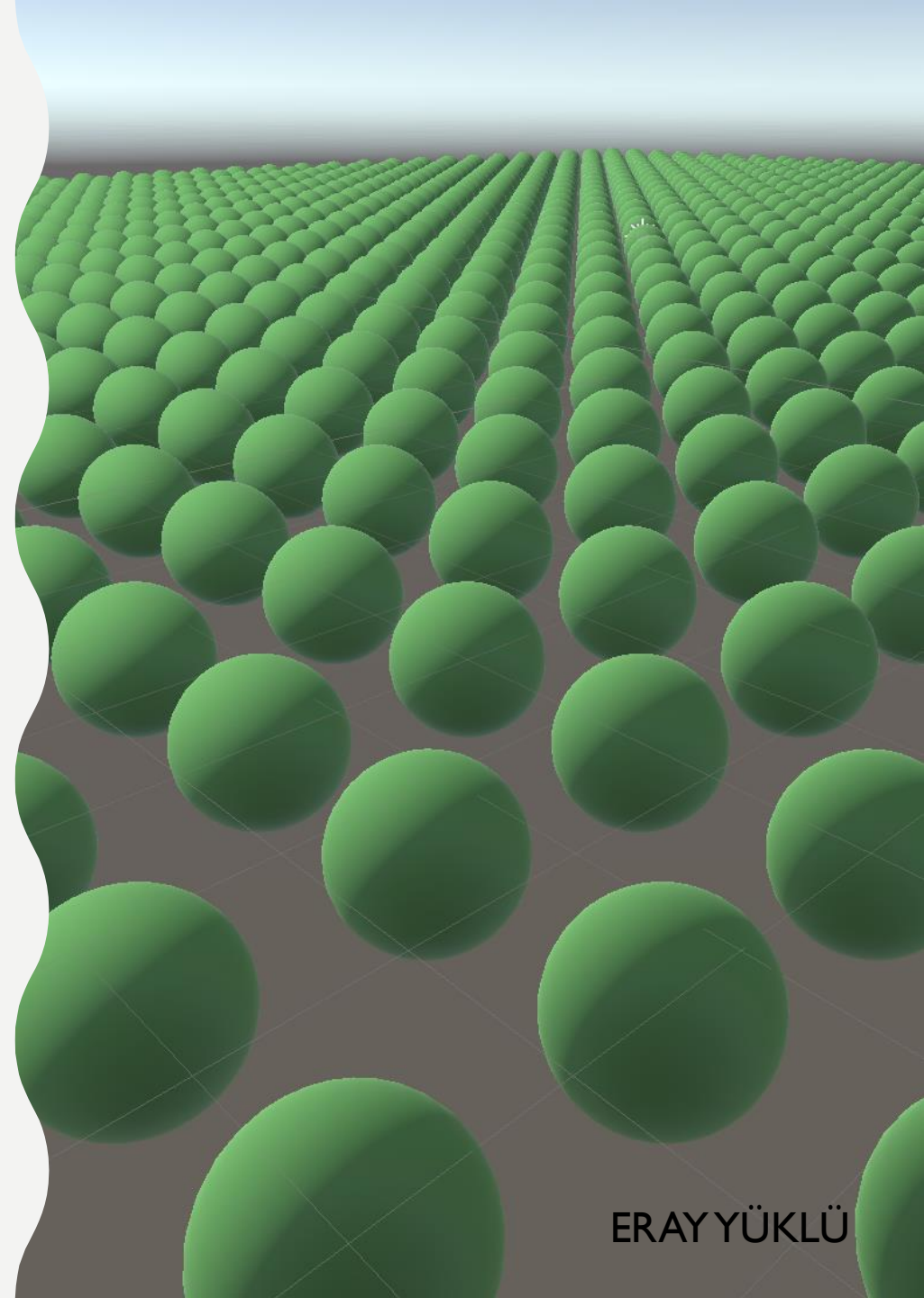
WHY BATCHING MATTERS

- **Every draw call is a CPU cost.** Modern scenes can issue thousands per frame.
- **CPU-bound rendering caps frame rate** long before the GPU is saturated.
- **Unity exposes two main remedies:** GPU Instancing and Static Batching.
- Each carries different **cost, memory, and flexibility trade-offs.**
- This project **quantifies** those trade-offs on a real Unity build.
- *Foundational reference: Pranckevičius, A. (2015). "Rendering Optimisation in Unity."*

EXPERIMENT DESIGN

A controlled $3 \times 3 \times 5$ sweep, repeated 3x

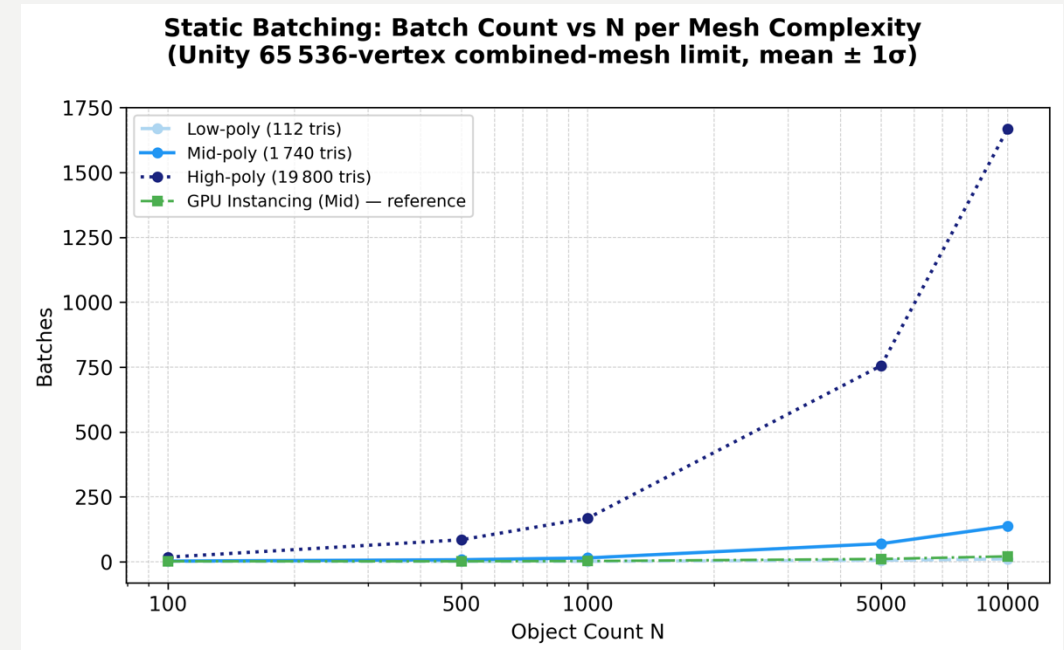
- **Rendering mode:** Baseline | GPU Instancing | Static Batching
- **Mesh complexity:** Low-poly (112 tris) | Mid-poly (1,740) | High-poly (19,800)
- **Object count N:** 100, 500, 1 000, 5 000, 10 000
- **Per cell:** 3 repeats with mean $\pm 1\sigma$ reporting → **135 configurations**
- **Recorded:** FPS, wall-clock + GPU frame time, draw call batches, reserved memory



CPU DRAW-CALL SUBMISSION → GPU VERTEX THROUGHPUT

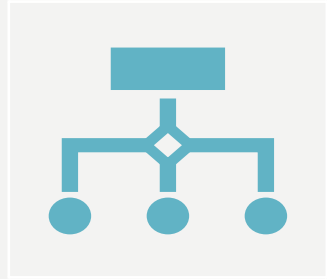
STATIC BATCHING BREAKS DOWN

- **Unity caps each combined mesh at 65,536 vertices**
 - High-poly (19,800 tris $\approx$ 10,000 verts): only $\sim$ 6 objects fit per batch
- **Result: batch count grows linearly with N at high-poly**
 - N=10,000 $\rightarrow$ 1,668 batches (vs. 21 with GPU Instancing)
 - Combined mesh also balloons reserved memory to 6.2 GB
- **GPU Instancing has no such cap — stays at $\sim$ 21 batches regardless of mesh complexity**



Static-batch count vs N — note vertex-cap cliff

METHODOLOGY & METRICS



Variables

- **Object count $N \in \{100, 500, 1000, 5000, 10000\}$**
- **Mesh complexity:** Low (112), Mid (1 740), High (19 800 tris)
- **Mode:** Baseline, GPU Instancing, Static Batching (3×5×3 = 45 cells × 3 repeats)



Metrics Captured

- **FPS & frame time** : wall-clock vs. GPU (FrameTimingManager)

Draw-call batches : Unity ProfilerRecorder (Batches Count); theoretical formula substituted when Metal returns 0

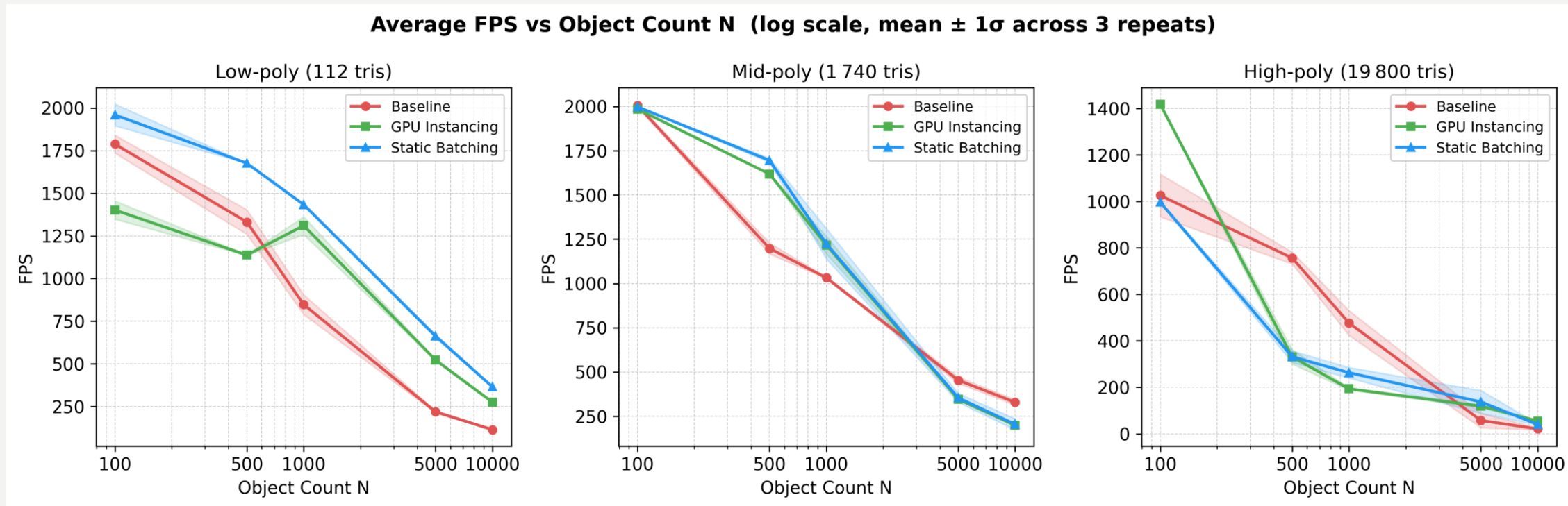
- **Total reserved memory:**

Profiler.GetTotalReservedMemoryLong()

- **Per cell:** 3 s warm-up discarded, then 30 s measurement window (×3 repeats = 135 windows)

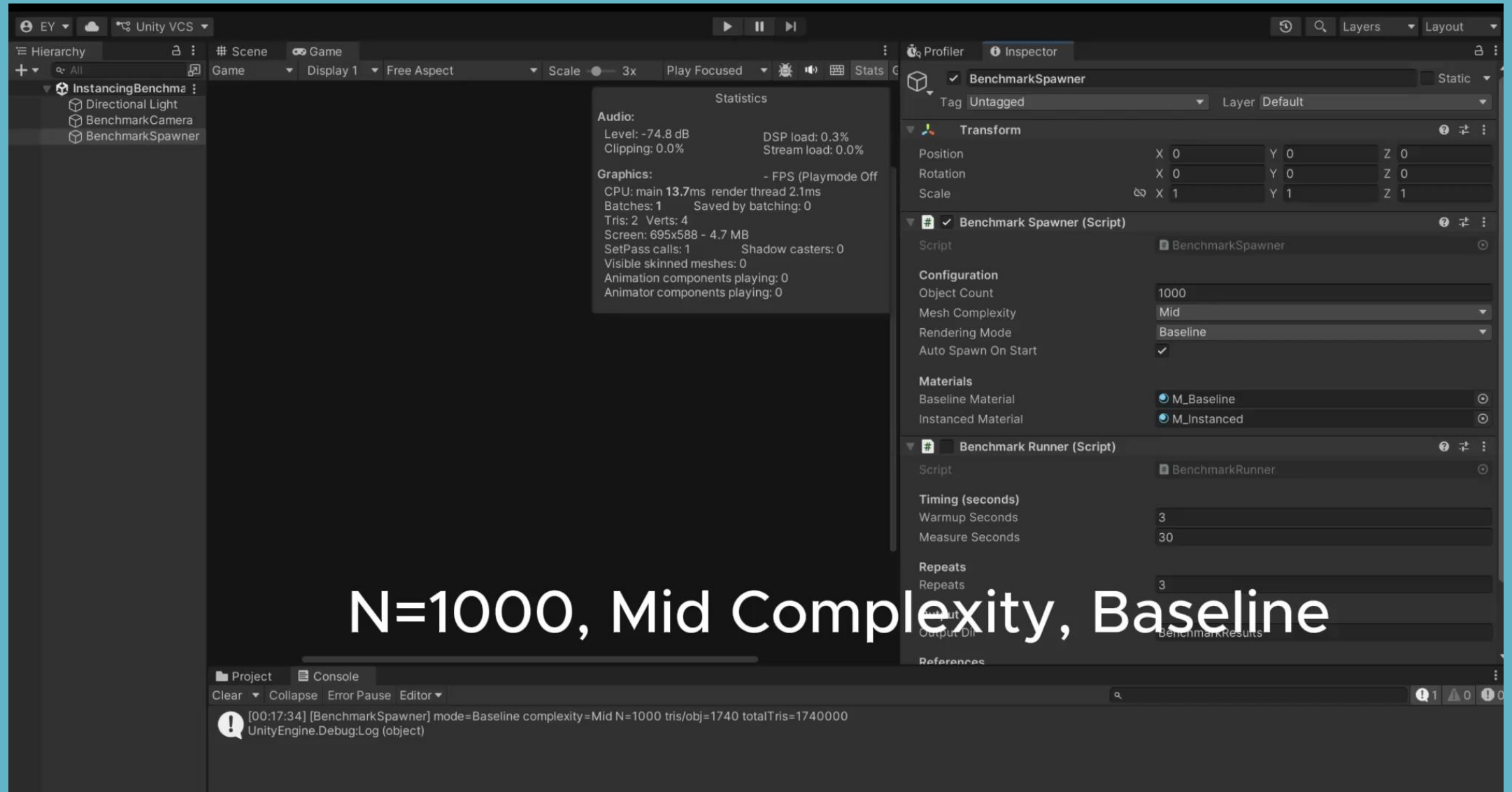
Cross-validated against Unity Editor Profiler — harness agrees within 0.5 ms across CPU- and GPU-bound regimes.

RESULT: GPU INSTANCING WINS AT SCALE



At N=10000 high-poly: Baseline 21 FPS $\rightarrow$ GPU Instancing 55 FPS (2.6 $\times$). Static Batching: 40 FPS (1.9 $\times$).

DEMO VIDEO: GPU INSTANCING IN ACTION



Video Link: <https://drive.google.com/file/d/1BxELsNZe8aG59iXXfvFj2ZmjgLVN-Sej/view?usp=sharing>

ERAY YÜKLÜ

CONCLUSIONS & TAKEAWAYS



GPU Instancing scales best. Constant batch count (≤ 21) regardless of N; 2.6× FPS gain at N=10,000 high-poly.



Static Batching has hidden costs. 6.2 GB memory at high-poly; vertex cap forces re-batching as mesh size grows.



Bottleneck shifts with mesh complexity. Low/Mid-poly: CPU-bound (batching wins). High-poly: GPU-bound (batching gains plateau).



Recommendation: Default to GPU Instancing for repeated meshes; reserve Static Batching for small, light scenes.



Future work. Extend to URP / HDRP + SRP Batchers; add frustum-culling exercise; cross-platform comparison (D3D11, Vulkan).